Laboratory Assessment I

PEC-2 Laboratory: Full Stack Development - BIT26PE05

Run a basic Java Maven project

Varad Sushant Deshmukh [123B1F022] TY-IT-A, PCCOE College

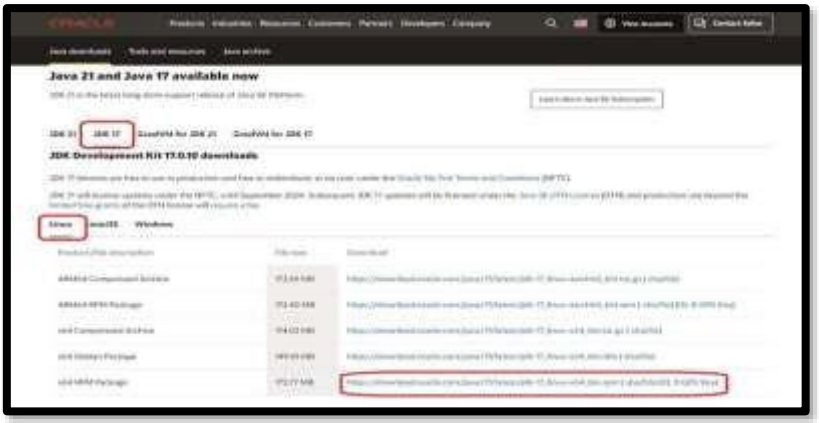
AIM:

To install JDK 17 and IntelliJ IDEA, and to create and run a basic Java Maven project and a Spring Boot application manually (without using Spring Initializr).

OBJECTIVES:

- To download and install JDK 17 and configure environment variables
- To install and set up IntelliJ IDEA IDE
- To understand how to create a Maven project structure
- To manually configure Spring Boot using pom.xml
- To develop and run a basic Hello World application

STEP1: Download JDK 17



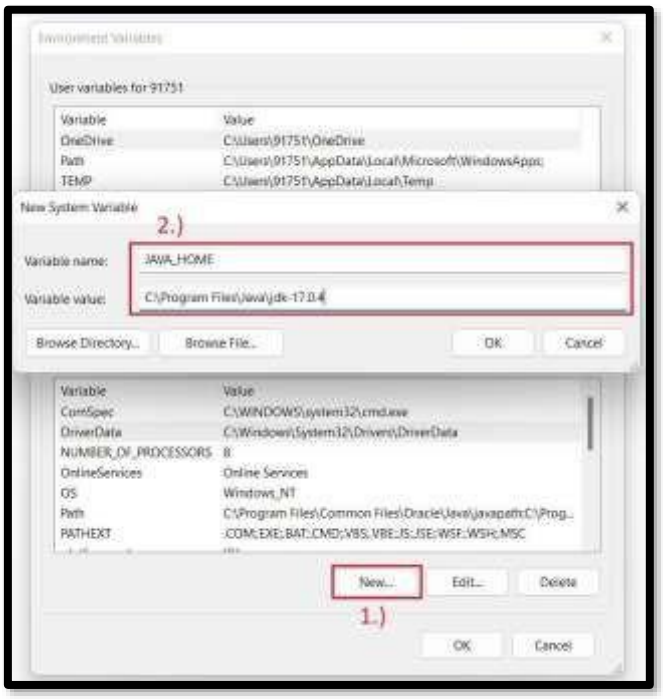
Step 2: Run Installer

Step 3: Path setting

Click **Edit the system environment variables**

- Click **New (System Variables)**
- Name: JAVA_HOME
- Value:

C:\Program Files\Java\jdk-17



Step4: Verify Installation: Run in Command Prompt:

java -version javac -version

```
C:\Users\Arpita>java -version
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_482-b08)
OpenJDK 64-Bit Server VM (Temurin)(build 25.482-b08, mixed mode)
```

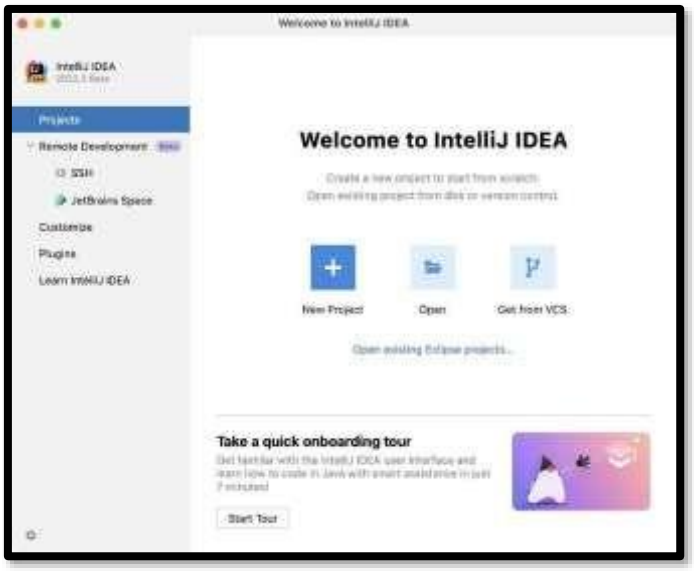
Step 1: Download IntelliJ

Go to: https://www.jetbrains.com/idea/download

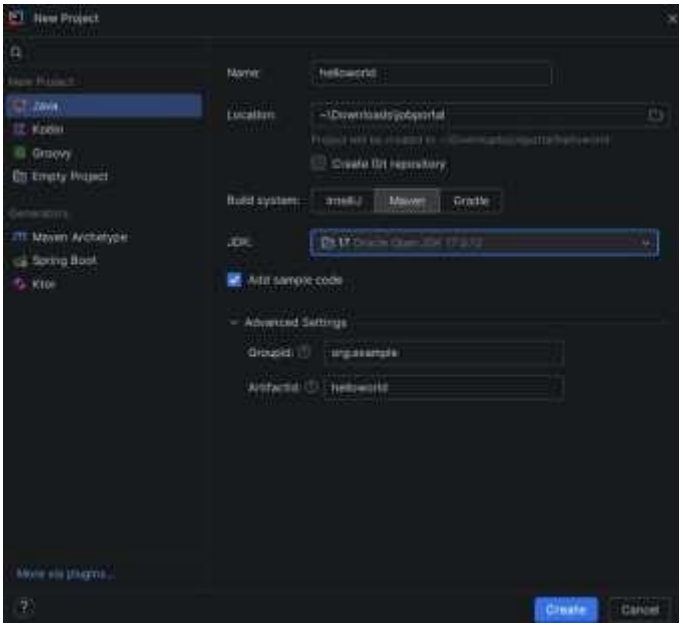
Step 2: Run installer Select:

- Add to PATH
- Create Desktop Shortcut

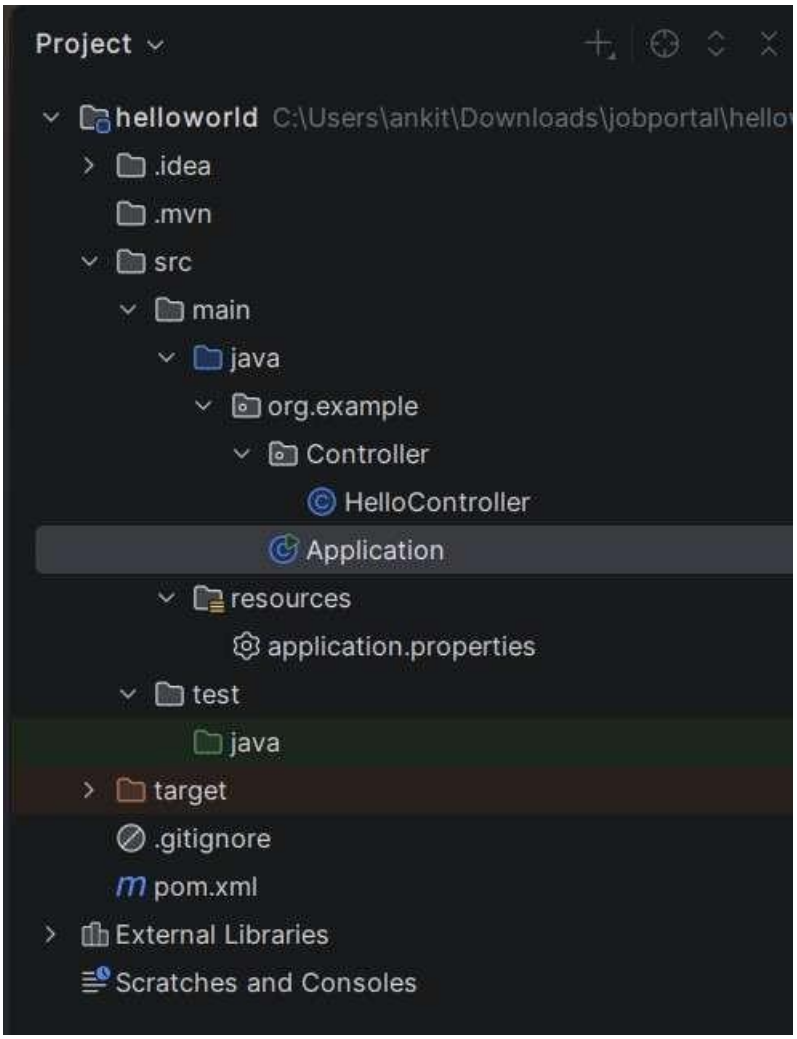
Click **Next → Install → Finish**



Create spring boot Hello World App:



Project Structure:



Application.java

The Application class is the entry point of the Spring Boot project. It contains the main() method and uses @SpringBootApplication to enable auto-configuration, component scanning, and configuration setup. It starts the embedded server (Tomcat) and runs the application.

```
package org.example; import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;


@SpringBootApplication public class Application {

   public static void main(String[] args) {

                  SpringApplication.run(Application.class, args);

   }

}
```

HelloController.java

HelloController.java is a REST controller class that handles HTTP requests. It uses annotations like @RestController and @GetMapping to define endpoints. When the application runs, it returns a simple response (e.g., “Hello World”) when accessed through a web browser.

```
package org.example.Controller; import
org.springframework.web.bind.annotation.GetMapping; import
org.springframework.web.bind.annotation.RestController;


@RestController public class HelloController {    @GetMapping("/")    public
String hello() {        return "Hello World";

}

}
```

pom.XML

pom.xml (Project Object Model) is the core configuration file of a Maven project. It defines:

- Project details (groupId, artifactId, version)
- Dependencies (Spring Boot libraries)
- Build configuration and plugins

It controls how the project is built and which libraries are included.

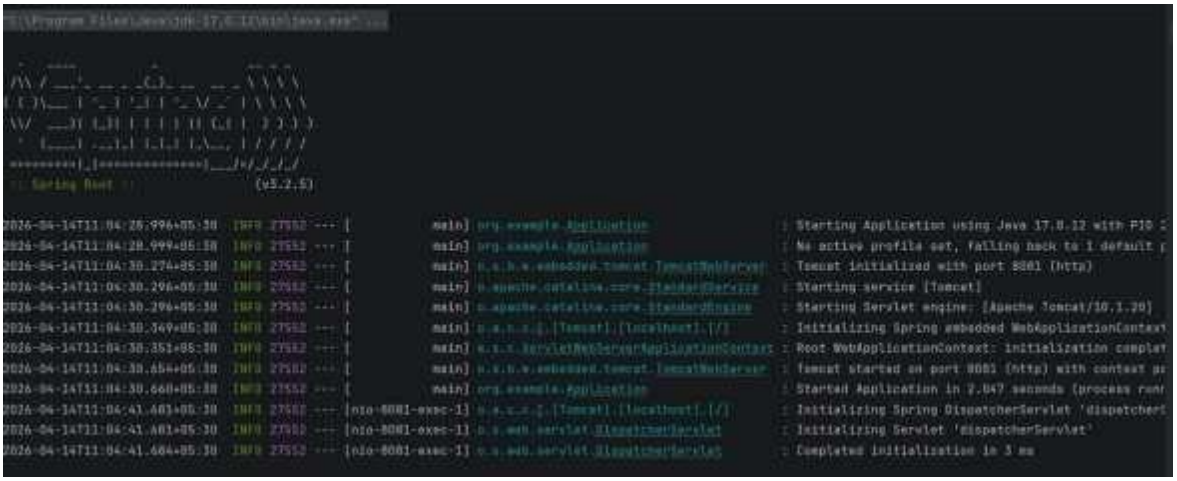
```
<?xml version="1.0" encoding="UTF-8"?> <project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>springboot-manual</artifactId>
    <version>1.0-SNAPSHOT</version>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
    </parent>
    <dependencies>
            <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

OUTPUT:



Conclusion:

The assignment demonstrated the complete setup of the Java development environment using **JDK 17** and **IntelliJ IDEA**. A Maven-based project was successfully created, and a Spring Boot application was built manually without relying on Spring Initializr, providing a deeper understanding of project configuration and dependencies. The application was executed successfully.
